Lektion 9

Proces-synkronisering

Computerarkitektur og Operativsystemer



Emil Lykke Diget

Syddansk Universitet

eld@mmmi.sdu.dk

Lektionsoversigt

- Lektion 1: Kombinatoriske Logiske Kredse
- Lektion 2: En Simpel Computer
- Lektion 3: Hukommelse
- Lektion 4: Mikroarkitektur
- Lektion 5: Micro-assembly og IJVM
- Lektion 6: Optimering af Mirkoarkitekturdesign
- Lektion 7: Introduktion til operativsystemer
- Lektion 8: Tråde og CPU-schedulering
- Lektion 9: Processynkronisering
- Lektion 10: Deadlocks
- Lektion 11: Memory management
- Lektion 12: Virtuel hukommelse

Outline

Motivation
- Bounded Buffer
- Processynkronisering

Semaforer
- Semafor-kø

Deadlocks

Klassiske Synkroniseringsproblemstillinger
- *Bounded-Buffer*-problem
- *Readers-Writers*-problem
- *Dining-Philosphores*-problem

Monitorer
- En monitor-løsning på *Dining Philosophors*

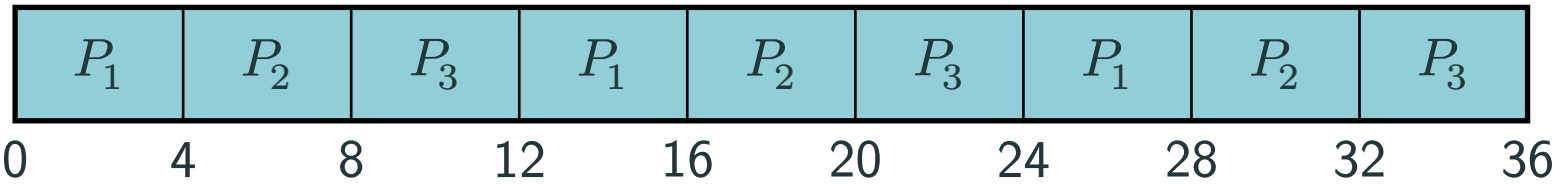
Motivation

Motivation

Ved preemptiv schedulering kan processer bliver afbrudt på hvilket som helst tidspunkt.

Dette kan give problemer, når processer samarbejder om forskellige arbejdsopgaver.

Ses f.eks. ved Round-Robin-schedulering (fra sidste lektion).



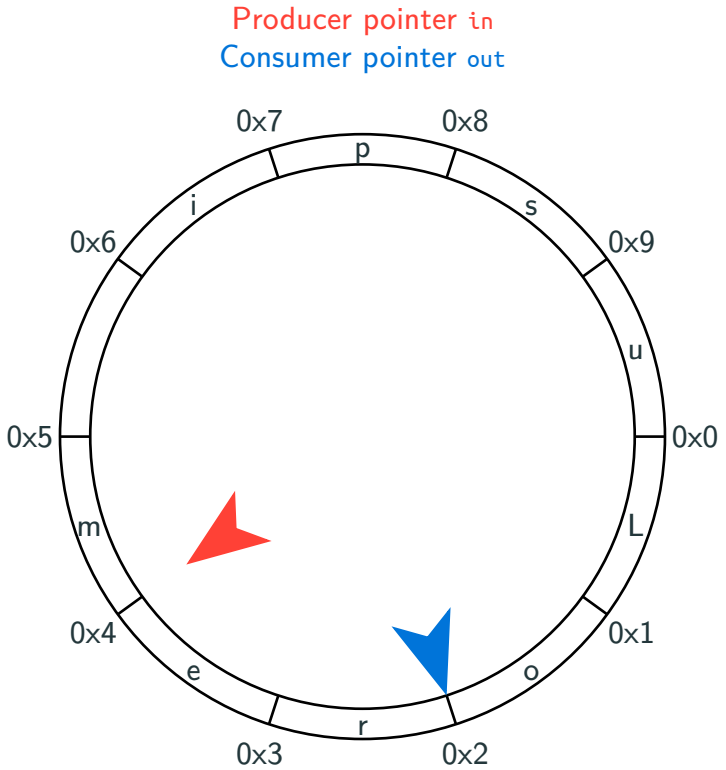
I dag skal vi kigge på forskellige scenarior, hvor disse problemer kan opstå.

Eksempel – Bounded Buffer

En cirkulær buffer tilgås af en *producer* og en *consumer*.

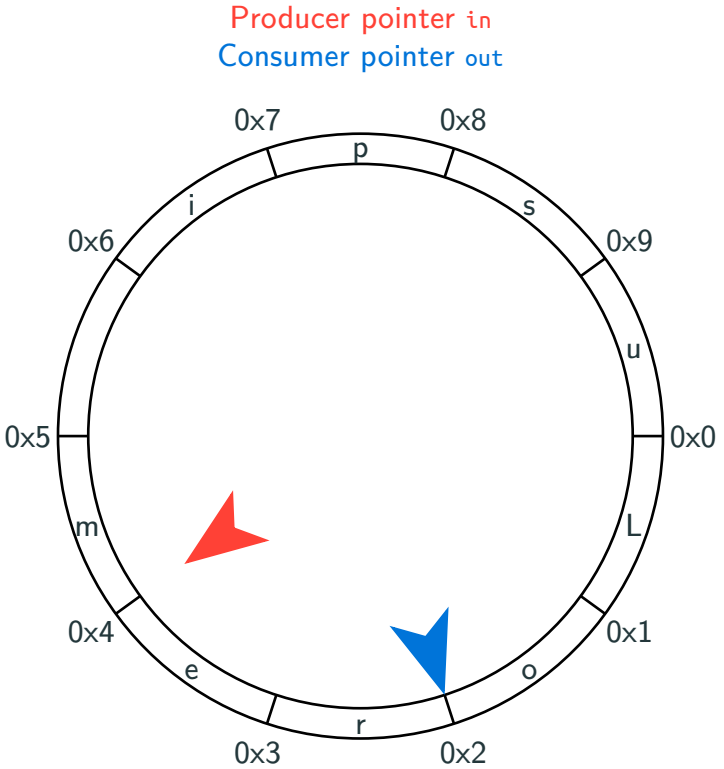
Producer

```
1  item next_produced;
2
3
4  while (true) {
5    /* produce an item in next_produced*/
6
7    while (count == BUFFER_SIZE) {} // do
nothing
8
9    buffer[in] = next_produced;
10   in = (in + 1) % BUFFER_SIZE;
11   count++;
12 }
```



Consumer

```
1  item next_consumed;
2
3  while (true) {
4    while (count == 0) {} // do nothing
5
6    next_consumed = buffer[out];
7    out = (out + 1) % BUFFER_SIZE;
8
9    count--;
10
11   /* consume an item in next_consumed*/
12 }
```



Eksempel – Bounded Buffer

Her har vi en **race condition**

`count++` vil blive udført som:

```
1  register1 = count
2  register1 = count + 1
3  count = register1
```

Eksempel – Bounded Buffer

Her har vi en **race condition**

`count++` vil blive udført som:

```
1  register1 = count
2  register1 = count + 1
3  count = register1
```

`count--` vil blive udført som:

```
1  register2 = count
2  register2 = count - 1
3  count = register2
```

Eksempel – Bounded Buffer

Der vil opstå problemer, hvis disse skulle køre samtidigt. `count = 5` i starten.

T_0 :	*producer*	execute	`register1 = count`	`{register1 = 5}`
T_1 :	*producer*	execute	`register1 = register1 + 1`	`{register1 = 6}`
T_2 :	*consumer*	execute	`register2 = count`	`{register2 = 5}`
T_3 :	*consumer*	execute	`register2 = register2 - 1`	`{register2 = 4}`
T_4 :	*producer*	execute	`count = register1`	`{count = 6}`
T_5 :	*consumer*	execute	`count = register2`	`{count = 4}`

Ved T_5 er `count = 4`, hvilket er forkert! Den burde være 5.

Hvis T_4 og T_5 var byttet, havde vi haft `count = 6`. Også forkert...

Dette kaldes en **race condition**, hvor processer – i et preemptivt miljø – har lov til at manipulere den samme variabel (`count`).

Løningen på dette er at synkronisere processerne; *proces-synkronisering*.

```
1  while (true) {
2      entrér sektion
3          den kritiske sektion
4
5      gå ud af sektion
6          resten af koden
7  }
```

The critcal section eller *den kritiske sektion* på dansk.

Givet et system med N processer $\{P_0, ..., P_{N-1}\}$, der alle har en kritisk sektion, hvor de manipulerer samme variabel.

En løsning skal opfylde de følgende tre krav:

1. ***Mutual exclusion***: Hvis process P_i manipulerer variabel x, må *ingen* anden process være i en kritisk sektion, der manipulerer samme variabel.
2. ***Progress***: Kun processer, der ønsker at indtræde deres kritiske sektion, og som ikke er i deres remainder sektion kan bestemme, hvilken process, der kan gå i kritisk sektion næste gang.
3. ***Bounded waiting***: Der eksisterer en øvre grænse på antal gange, en process skal vente, før den kan gå i kritisk sektor.

Semaforer

Semaforer

Definer en heltalsvariabel (integer) `S`, der *kun* kan manipuleres via to *atomiske* funktioner `wait()` og `signal()`.

```
1  int S = 1; // Semafor; hvad betyder init-værdien?
2
3  void wait(S)
4  { // Kaldes når en process ønsker adgang til kritisk sektion
5    while (S <= 0)
6      ; // busy wait
7    S--;
8  }
9
10 void signal(S)
11 { // Kaldes når en process ønsker at forlade sin kritiske sektion
12   S++;
13 }
```

Semaforer

Definer en heltalsvariabel (integer) `S`, der *kun* kan manipuleres via to *atomiske* funktioner `wait()` og `signal()`.

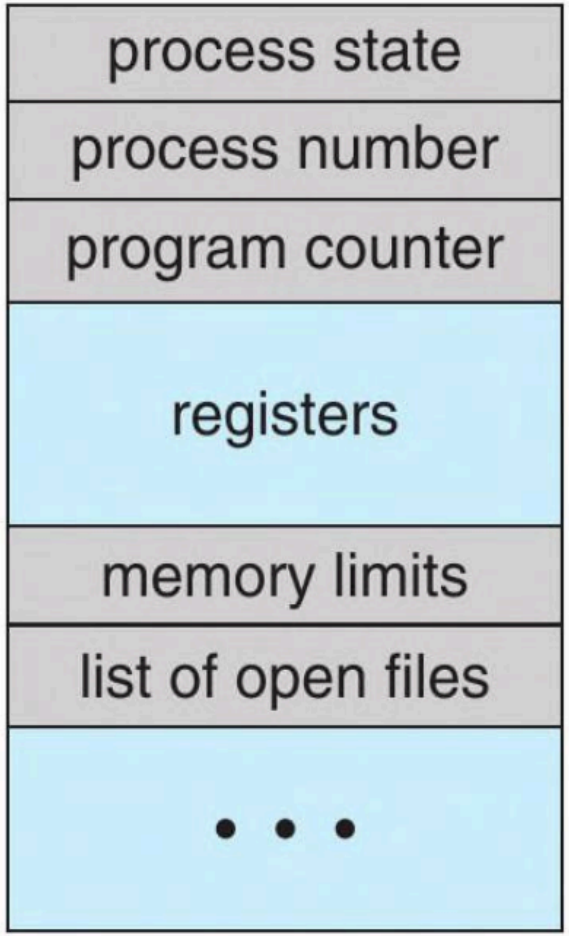
```
1  int S = 1; // Semafor; hvad betyder init-værdien?
2
3  void wait(S)
4  { // Kaldes når en process ønsker adgang til kritisk sektion
5    while (S <= 0)
6      ; // busy wait
7    S--;
8  }
9
10 void signal(S)
11 { // Kaldes når en process ønsker at forlade sin kritiske sektion
12   S++;
13 }
```

Busy wait kan være problematisk, da CPUen da bruger tid på bare at vente.

Semafor-kø

Husk på *process control block* (PCB) fra lektion 7; vi skal bruge *process state*-feltet.

Koncept: Lav en semafor-kø, hvor en process kan vente, hvis den ikke kan få semaforen med det samme.



Semafor-kø

En semafor kan defineres med en `value` og en kø `list` til ventende processer.

```
1  typedef struct {
2    int value;
3    struct process *list;
4  } semaphore;
5
6  semaphore S;
7  S.value = 1;
8  process P1, P2; // etc.
```

Semafor-kø

En semafor kan defineres med en `value` og en kø `list` til ventende processer.

```
1  typedef struct {
2    int value;
3    struct process *list;
4  } semaphore;
5
6  semaphore S;
7  S.value = 1;
8  process P1, P2; // etc.
```

```
1  void wait(semaphore *S, process *P) {
2    S->value--;
3    if (S->value < 0) {
4      S->list.add(P);
5      P->sleep(); // suspenderer processen
6    }
7  }
8
9  void signal(semaphore *S, process *P) {
10   S->value++;
11   if (S->value <= 0) {
12     S->list.remove(P);
13     P->wakeup();
14   }
15 }
```

Semafor-kø

En semafor kan defineres med en `value` og en kø `list` til ventende processer.

```
1  typedef struct {
2    int value;
3    struct process *list;
4  } semaphore;
5
6  semaphore S;
7  S.value = 1;
8  process P1, P2; // etc.
```

Hvad betyder en negativ værdi af `S.value`?

```
1  void wait(semaphore *S, process *P) {
2    S->value--;
3    if (S->value < 0) {
4      S->list.add(P);
5      P->sleep(); // suspenderer processen
6    }
7  }
8
9  void signal(semaphore *S, process *P) {
10   S->value++;
11   if (S->value <= 0) {
12     S->list.remove(P);
13     P->wakeup();
14   }
15 }
```

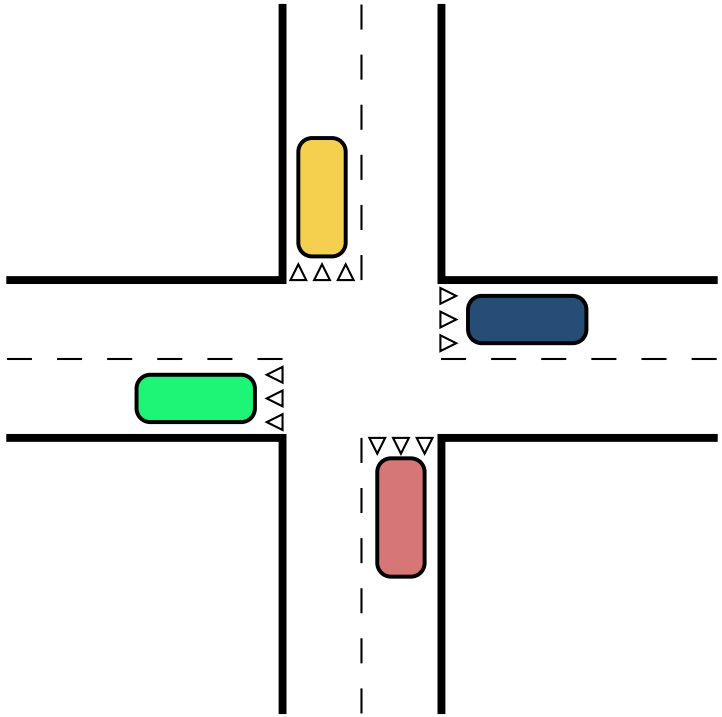
Deadlocks

På dansk: *Hårdknuder*.

Definition (*Hårdknude*) Venter en proces på frigivelsen af en en ressource, der kun kan frigives af en proces, der selv venter, kaldes det en hårdknude (*deadlock*).

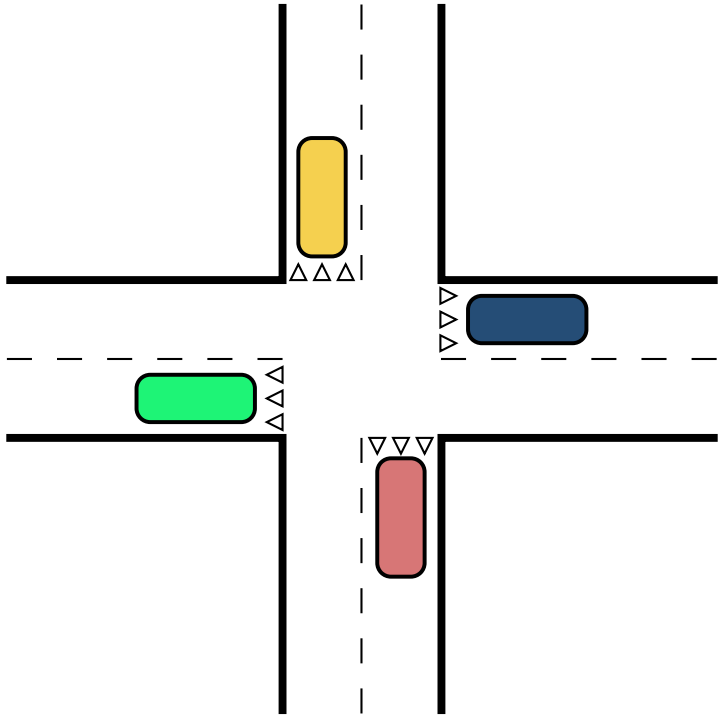
Deadlocks

I kan tænke på det som et ureguleret vejkryds, hvor højre vigepligt gælder.



Deadlocks

I kan tænke på det som et ureguleret vejkryds, hvor højre vigepligt gælder.



... og bilisterne kan ikke vinke til hinanden (synkroniseringssignaler).

Hvem kører først?

Deadlocks

Som kode kan det se således ud:

```
1  semaphore S, Q;
```

P_0

```
1  wait(S);
2  wait(Q);
3  // content
4  signal(S);
5  signal(Q);
```

P_1

```
1  wait(Q);
2  wait(S);
3  // content
4  signal(Q);
5  signal(S);
```

Hvad er problemet i ovenstående kode?

Det kan være lumsk; det kan se rigtigt ud på papiret, men alligevel føre til deadlock.

Deadlocks

Som kode kan det se således ud:

```
1  semaphore S, Q;
```

P_0

```
1  wait(S);
2  wait(Q);
3  // content
4  signal(S);
5  signal(Q);
```

P_1

```
1  wait(Q);
2  wait(S);
3  // content
4  signal(Q);
5  signal(S);
```

Hvad er problemet i ovenstående kode?

Det kan være lumsk; det kan se rigtigt ud på papiret, men alligevel føre til deadlock.

En tommelfingerregel: Kald `wait` og `signal` i en bestemt rækkefølge på tværs af processer.

Klassiske Synkroniseringsproblemstillinger

Klassiske Synkroniseringsproblemstillinger

Vi skal kigge på de følgende tre klassiske problemstillinger:

- *Bounderd-buffer*-problem
- *Readers and writer*-problem
- *Dining-philosophors*-problem

Det er en række problemstillinger, der bruges til at teste synkroniseringsmekanismer.

Bounded-Buffer-problem

Husk tilbage på producer-consumer-problemet fra starten af slideshowet.

Delte datastrukturer:

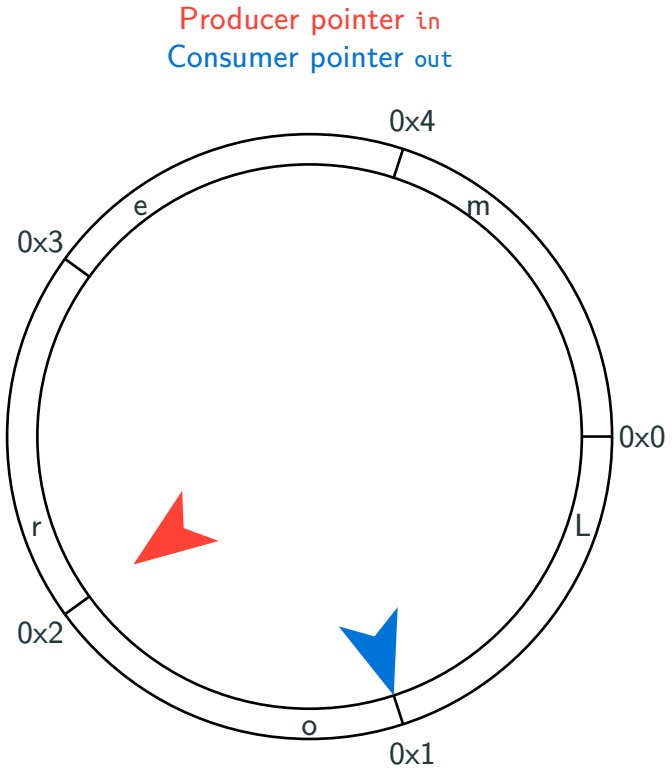
```
1  int buffer[5];
2  int n = 5;
3  semaphore mutex = 1;
4  semaphore empty = n;
5  semaphore full = 0;
```

`mutex` giver adgang til buffer-poolen med start-værdi 1.

`empty` tæller antallet af tomme buffere.

`full` tæller antallet af fulde buffere.

Producer tilføjer ting til buffer, og *consumer* tager ting ud.



Producer-procesen

```
1  while (true)
2  {
3      . . .
4    /* producer ny data og læg i next_produced */
5    wait(empty);  // vent på plads i buffer
6    wait(mutex);  // vent på adgang til buffer
7      . . .
8    /* flyt data fra next_produced til buffer */
9      . . .
10   signal(mutex);  // giv adgang tilbage til buffer
11   signal(full);  // tæl full 1 op
12     . . .
13 }
```

Consumer-procesen

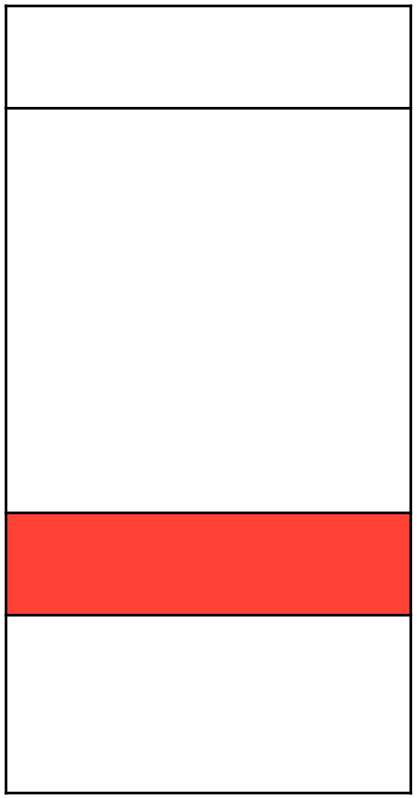
```
1  while (true)
2  {
3    wait(full);   // vent på data, altså full > 0
4    wait(mutex);  // vent på adgang til buffer
5      . . .
6    /* flyt data fra buffer til next_consumed */
7      . . .
8    signal(mutex);  // giv adgang tilbage til buffer
9    signal(empty);  // tæl empty 1 op
10     . . .
11   /* brug data i next_consumed */
12     . . .
13 }
```

Readers-Writers-problem

Forestil jer en database delt af mange processer. Det kunne f.eks. være et delt område i hukommelsen.

Nogle processer vil skrive – andre vil læse;

- En *reader*-process vil gerne læse fra delt område.
- En *writer*-process vil gerne opdatere delt område.



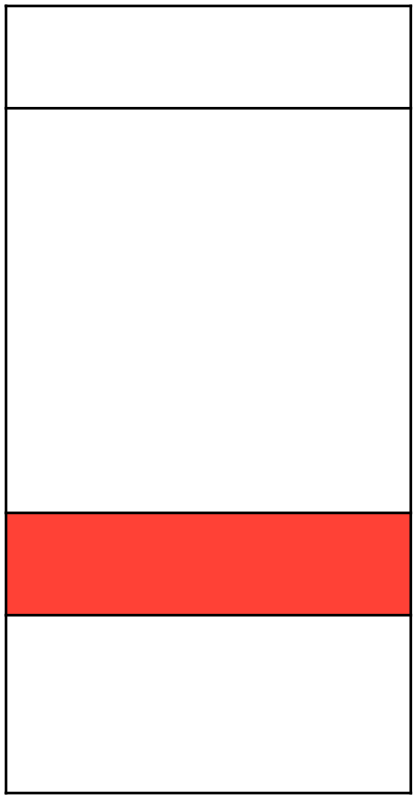
Readers-Writers-problem

Forestil jer en database delt af mange processer. Det kunne f.eks. være et delt område i hukommelsen.

Nogle processer vil skrive – andre vil læse;

- En *reader*-process vil gerne læse fra delt område.
- En *writer*-process vil gerne opdatere delt område.

Intet problem: To *reader* tilgår område samtidigt.



Readers-Writers-problem

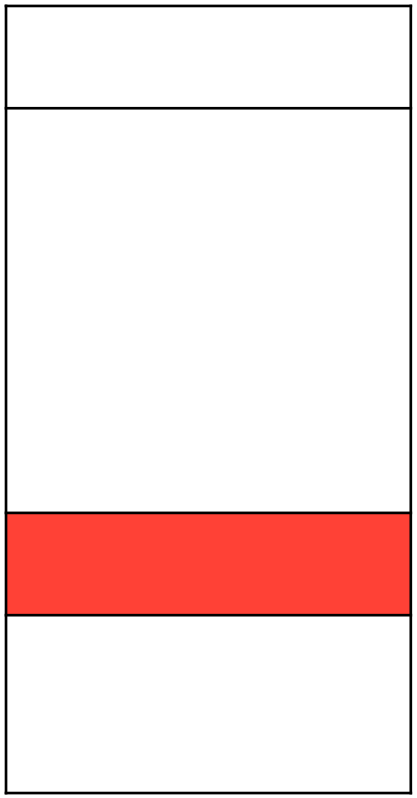
Forestil jer en database delt af mange processer. Det kunne f.eks. være et delt område i hukommelsen.

Nogle processer vil skrive – andre vil læse;

- En *reader*-process vil gerne læse fra delt område.
- En *writer*-process vil gerne opdatere delt område.

Intet problem: To *reader* tilgår område samtidigt.

Problem: En *writer* og en anden (*reader* el. *writer*) ønsker at tilgå området samtidigt.



Readers-Writers-problem

Bogen skelner mellem to slags *readers-writers*-problem:

1. *First readers-writers*-problem:

   En *reader* må kun vente, hvis en *writer* har fået adgang til den fælles ressource.

2. *Second readers-writers*-problem:

   Hvis en *writer* er klar til at skrive, må flere *readers* ikke begynde at læse.

Det kommer an på prioriteringen mellem *reader* og *writer*.

Readers-Writers-problem

Bogen skelner mellem to slags *readers-writers*-problem:

1. *First readers-writers*-problem:

   En *reader* må kun vente, hvis en *writer* har fået adgang til den fælles ressource.

2. *Second readers-writers*-problem:

   Hvis en *writer* er klar til at skrive, må flere *readers* ikke begynde at læse.

Det kommer an på prioriteringen mellem *reader* og *writer*.

Vi følger bogen og kigger på den første.

Readers-Writers-problem

Delte ressourcer:

```
1  semaphore rw_mutex = 1;  // Mutex-semafor for writers; kun én kan skrive af gangen.
2    // Bruges også af den første og den sidste reader, der går ind i den kritiske sektor.
3
4  semaphore mutex = 1;     // Fungerer som semafor omkring read_count mens den opdateres.
5
6  int read_count = 0;      // Antal processer der læser det fælles objekt.
```

Readers-Writers-problem

Writer

```
1  while (true)
2  {
3    wait(rw_mutex);
4      . . .
5    // Udfør skrivning
6      . . .
7    signal(rw_mutex);
8  }
```

Readers-Writers-problem

Writer

```
1  while (true)
2  {
3    wait(rw_mutex);
4      . . .
5    // Udfør skrivning
6      . . .
7    signal(rw_mutex);
8  }
```

Reader

```
1  while (true)
2  {
3    wait(mutex);
4    read_count++;
5    // Er det første reader?
6    if (read_count == 1)
7      wait(rw_mutex);
8    signal(mutex);
9      . . . // Udfør læsning
10   wait(mutex);
11   read_count--;
12   // Er det den sidste reader?
13   if (read_count == 0)
14     signal(rw_mutex);
15   signal(mutex);
16 }
```

Readers-Writers-problem

Writer

```
1  while (true)
2  {
3    wait(rw_mutex);
4      . . .
5    // Udfør skrivning
6      . . .
7    signal(rw_mutex);
8  }
```

Så længe bare én *reader* er i systemet, kan *writer* ikke skrive.

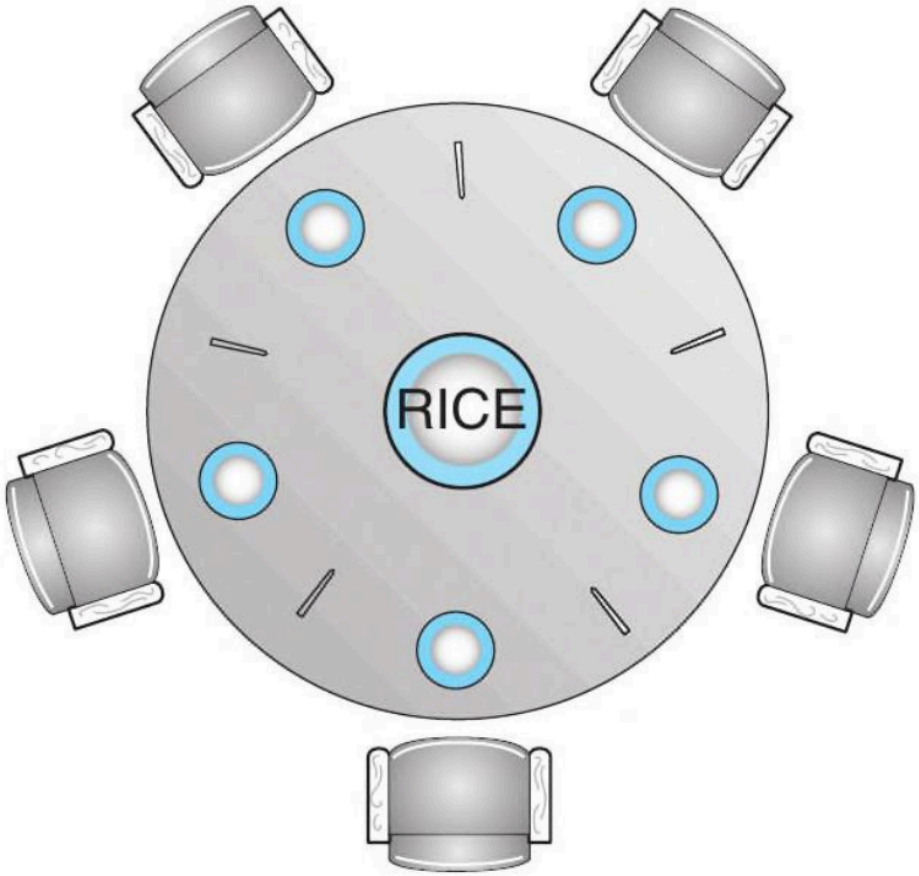
Reader

```
1  while (true)
2  {
3    wait(mutex);
4    read_count++;
5    // Er det første reader?
6    if (read_count == 1)
7      wait(rw_mutex);
8    signal(mutex);
9      . . . // Udfør læsning
10   wait(mutex);
11   read_count--;
12   // Er det den sidste reader?
13   if (read_count == 0)
14     signal(rw_mutex);
15   signal(mutex);
16 }
```

Dining-Philosphores-problem

Forestil jer følgende:

- Fem filosoffer der ikke laver andet end at *tænke* og *spise*.
- Delte ressourcer:
  - En stor skål ris i midten.
  - Fem spisepinde; én til højre, én til venstre for hver filosof.
- For at spise skal en filosof samle begge spisepinde op.



Delte ressourcer

```
1  semaphore chopstick [5];
2
3  for (int i = 0; i < 5; ++i)
4    chopstick[i] = 1;
```

Delte ressourcer

```
1  semaphore chopstick [5];
2
3  for (int i = 0; i < 5; ++i)
4    chopstick[i] = 1;
```

Filosof i

```
1  while (true)
2  {
3    // Saml to spisepinde op
4    wait(chopsticks[i]);
5    wait(chopsticks[(i + 1) % 5]);
6    // Spis i noget tid
7    signal(chopsticks[i]);
8    signal(chopsticks[(i + 1) % 5]);
9    // Tænk i noget tid
10 }
```

Dining-Philosphores-problem

Delte ressourcer

```
1  semaphore chopstick [5];
2
3  for (int i = 0; i < 5; ++i)
4    chopstick[i] = 1;
```

Problemer?

Filosof i

```
1  while (true)
2  {
3    // Saml to spisepinde op
4    wait(chopsticks[i]);
5    wait(chopsticks[(i + 1) % 5]);
6    // Spis i noget tid
7    signal(chopsticks[i]);
8    signal(chopsticks[(i + 1) % 5]);
9    // Tænk i noget tid
10 }
```

Dining-Philosphores-problem

Delte ressourcer

```
1  semaphore chopstick [5];
2
3  for (int i = 0; i < 5; ++i)
4    chopstick[i] = 1;
```

Problemer?

- Deadlock kan opstå:
  - Hvis alle tager `chopstick[i]` op samtidigt, skal alle vente uendeligt på `chopstick[(i + 1) % 5]`

Filosof i

```
1  while (true)
2  {
3    // Saml to spisepinde op
4    wait(chopsticks[i]);
5    wait(chopsticks[(i + 1) % 5]);
6    // Spis i noget tid
7    signal(chopsticks[i]);
8    signal(chopsticks[(i + 1) % 5]);
9    // Tænk i noget tid
10 }
```

Problem med Semaforer

Det kan være nemt at introducere bugs i koden f.eks:

```
1  signal(mutex);
2
3  // kritisk sektion
4
5  wait(mutex)
```

Hvad er galt her?

Problem med Semaforer

Det kan være nemt at introducere bugs i koden f.eks:

```
1  signal(mutex);
2
3  // kritisk sektion
4
5  wait(mutex)
```

Hvad er galt her?

```
1  wait(mutex);
2
3  // kritisk sektion
4
5  wait(mutex)
```

Hvad er galt her?

Monitorer

Monitorer

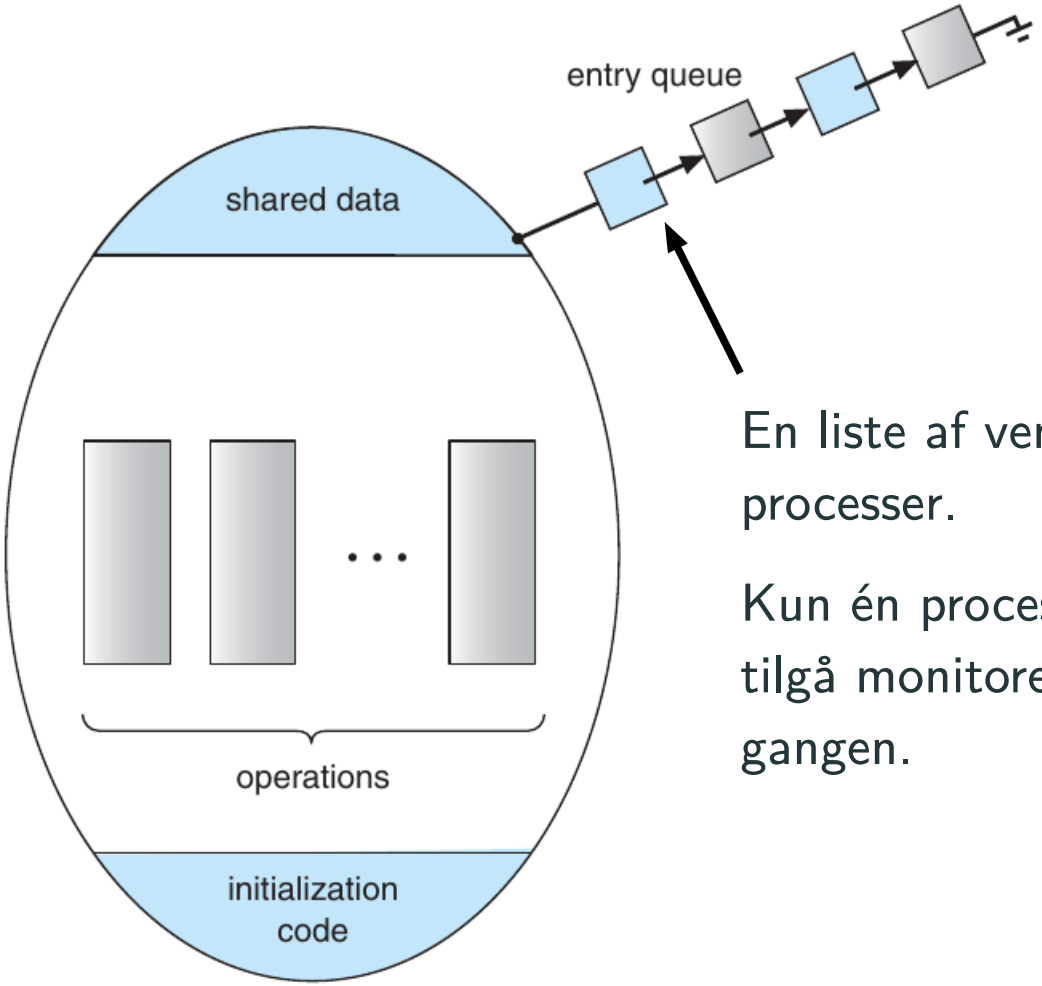
Med en bestemt konstruktion kaldet en **monitor** kan programmøren undgå de simple bugs, når man arbejder med semaforer.

Pseudokode:

```
1  monitor MonitorTemplate
2  {
3    // Deklaration af delte variable
4    function P1 (...) {...}
5    function Pn (...) {...}
6    function init_monitor (...) {...}
7  }
```

- En funktion defineret i monitoren kan *kun* tilgå lokale variable.

Monitorer

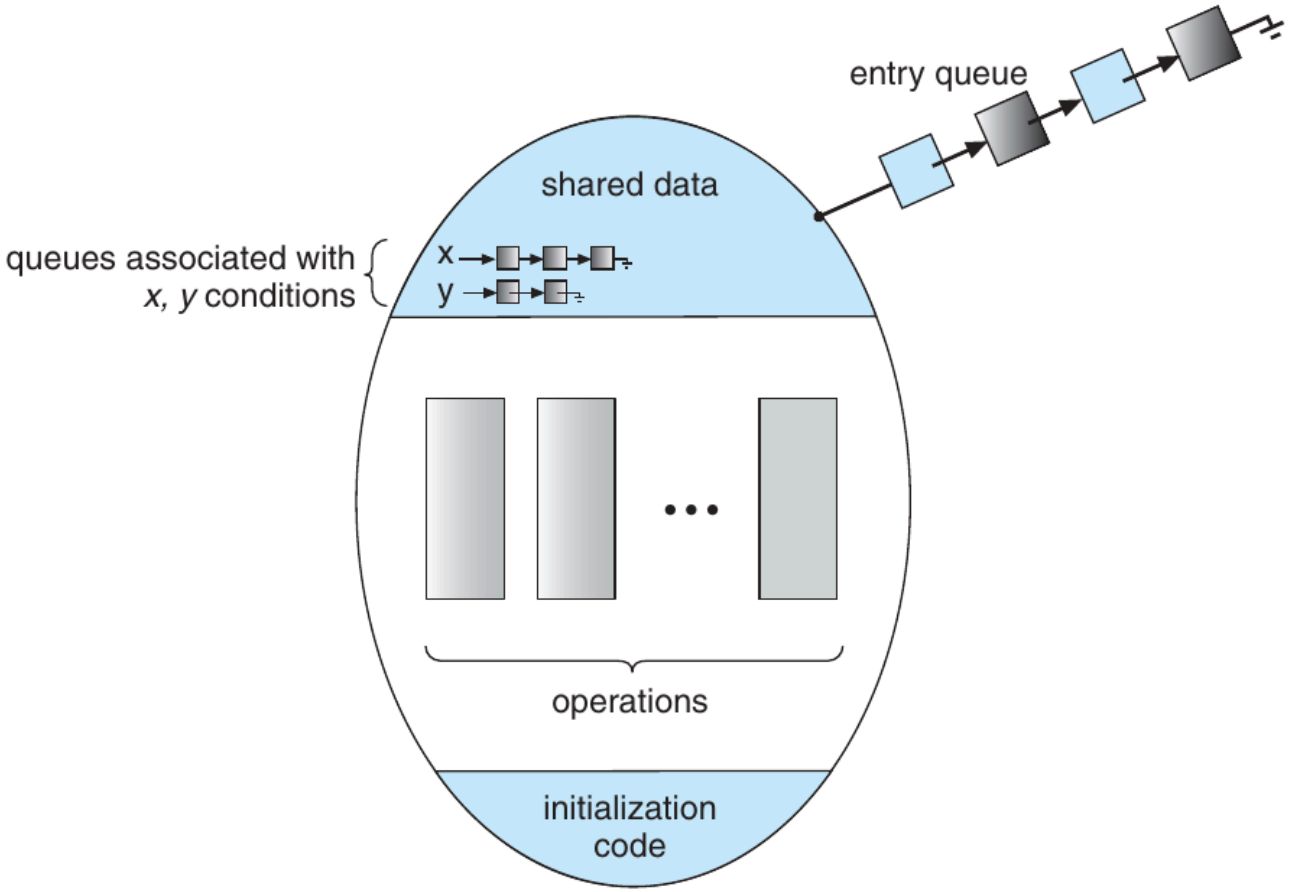


En liste af ventende processer.

Kun én process kan tilgå monitoren af gangen.

Monitorer

- Tilføjelse af *condition*-type.
- `x.wait()`: Process suspenderes, indtil en anden kalder `x.signal()`.
- `x.signal()`: Aktiverer en suspenderet process, hvis der er nogen.



En monitor-løsning på *Dining Philosophors*

```
1  monitor DiningPhilosphors
2  {
3    enum {THINKING, HUNGRY, EATING}
4      state[5];
5    condition self[5];
6    void pickup(int i) {
7      state[i] = HUNGRY;
8      test(i); // kan jeg spise?
9      if (state[i] != EATING)
10       self[i].wait();
11   }
12   void putdown(int i) {
13     state[i] = THINKING;
14     test((i + 4) % 5);
15     test((i + 1) % 5);
16     // kan dem ved siden af spise nu?
17   }
```

```
1    void test(int i) {
2      if ((state[i + 4] % 4) != EATING) &&
3          (state[i] == HUNGRY) &&
4          (state[(i + 1) % 5] != EATING))
5      {
6        state[i] = EATING;
7        self[i].signal();
8      }
9    }
10
11   void init() {
12     for (int i = 0; i < 5; i++)
13       state[i] = THINKING;
14   }
15 }
```

En monitor-løsning på *Dining Philosophors*

Filosof `i` kan så tilgå monitoren således:

```
1  DiningPhilosphors.pickup(i);
2  // spis
3  DiningPhilosphors.putdown(i);
```

Løsningen med monitorer er deadlock-fri, men en filosof kan godt sulte ihjel (*starvation*).

Vi undgår copy-paste-bugs i koden, men en monitor kan ikke mere end semaforer kan.

Ansvaret for synkroniseringen ligger nu hos *monitoren* og ikke i *applikationen*.